Prometheus da Central ESP32-2432S028

Configuracao pela interface web

1. Abra o painel da Central na **porta 80** e clique em **Configurar Prometheus**, ou acesse `http://IP_DA_CENTRAL/prometheus`.
2. Informe o endereco do servidor Prometheus: IPv4 ou DNS, sem `http://`, caminho ou porta, por exemplo `192.168.1.10` ou `prometheus.local`.
3. Informe a **porta de metricas da Central**: 1..65535, padrao **9120**.
4. Informe o intervalo sugerido de coleta: 5..3600 segundos, padrao **15**.
5. Salve nas Preferences. A mudanca de porta e aplicada cerca de um segundo depois, sem reboot; a interface web nao muda de porta.
6. Baixe `/prometheus.yml`, adicione o job no servidor e recarregue a configuracao.

O modelo e **pull**: o Prometheus consulta `http://IP_DA_CENTRAL:PORTA/metrics`. Nao ha envio ativo, Pushgateway ou POST para o Prometheus. O endereco do servidor e uma referencia persistida: nao altera o servidor nem restringe quem pode coletar. O intervalo salvo so passa a valer quando aplicado no YAML do Prometheus. A porta do servidor Prometheus (normalmente 9090) e independente da porta de metricas da Central.

Servico	Porta
Painel, configuracao Wi-Fi, APIs, log, formulario Prometheus e download YAML	**80**, fixa
`/metrics` em formato Prometheus 0.0.4	Configurada e persistida, padrao **9120**

Se escolher 80, `/metrics` compartilha o servidor do painel, sem criar um segundo listener nessa porta. Nas demais portas, `/metrics` retorna 404 na 80 e o servidor de metricas atende somente a coleta, nao as paginas de configuracao. A porta antiga deixa de atender apos aplicar a mudanca. Essas regras valem em STA, no AP **ESP32_CENTRAL_CONFIG** (`192.168.4.1`) e apos reiniciar.

Os endpoints nao possuem autenticacao. Use rede confiavel e nao exponha a Central diretamente a Internet. Reserve o IP da Central no DHCP; gere o YAML novamente se o IP mudar. O download usa o IP STA quando conectado e o IP AP quando offline.

YAML do servidor

Substitua o IP e a porta pelos valores da Central:

```
scrape_configs:
  - job_name: 'esp32_central'
    scrape_interval: 15s
    metrics_path: /metrics
    static_configs:
      - targets: ['192.168.1.42:9120']
```

Se ja houver `scrape_configs:`, adicione apenas o job sem duplicar a secao. Use nomes de job unicos ou agrupe dispositivos nos targets.

Preferences/NVS

Namespace **`telemetry`**, separado de `wifi`:

- `settings`: registro versionado com endereco do servidor, porta e intervalo.
- `boots`: inicializacoes e resets por watchdog, gravados uma vez por boot.

As configuracoes sao carregadas antes de iniciar os listeners. Entrada invalida retorna HTTP **400** sem salvar. Falha ao gravar retorna **503**, mostra aviso na pagina e na Serial e preserva o estado ativo em RAM. Registros invalidos ou NVS indisponivel geram avisos na Serial; os padroes sao 9120 e 15 segundos. Se o registro de contadores for invalido ou nao puder ser gravado, suas series sao omitidas, com aviso no formulario e na Serial. Os contadores persistentes saturam em 32 bits. A limpeza do namespace `wifi` nao apaga `telemetry`; uma limpeza completa da NVS/flash por ferramentas externas pode apagar ambos.

Metricas reais publicadas

Todas as series possuem o label **`device`**, o MAC STA estavel da placa, inclusive no AP. Identificacoes textuais sao gauges com valor 1 e labels escapados. A resposta inclui `HELP` e `TYPE` uma vez por familia.

Grupo	Metrica	Semantica
Identificacao	`esp_info`	Valor 1; labels `chip` e `model`.
Identificacao	`esp_firmware_version`	Valor 1; labels `version` (mesma versao automatica dos comandos) e `build`.
Identificacao	`esp_ip_address`	Valor 1; labels `ip` e `interface` (`sta`/`ap`), por interface ativa.
Identificacao	`esp_mac_address`	Valor 1; MAC STA no label `mac`.
Disponibilidade	`esp_status_online`	1 com Wi-Fi STA conectado, 0 sem conexao; nao verifica Internet.
Disponibilidade	`esp_uptime_seconds`	Tempo monotono de 64 bits desde boot, em segundos.
Disponibilidade	`esp_last_restart_reason`	Valor 1; codigo `esp_reset_reason_t` no label `reason`.
Disponibilidade	`esp_boot_count`	Inicializacoes acumuladas na NVS.

Grupo	Metrica	Semantica
Wi-Fi	`esp_wifi_rssi_dbm`	RSSI em dBm; omitida sem STA conectado.
Wi-Fi	`esp_wifi_channel`	Canal; omitida sem STA conectado.
Wi-Fi	`esp_wifi_disconnect_count`	Eventos de desconexao desde boot, incluindo desconexoes deliberadas e tentativas.
Wi-Fi	`esp_wifi_tx_power`	Limite configurado no driver, em dBm; nao e consumo eletrico. Omitida sem STA ou se a leitura falhar.
CPU	`esp_cpu_frequency_mhz`	Frequencia atual em MHz.
CPU	`esp_cpu_usage_percent`	Uso agregado dos nucleos, de 0 a 100%, derivado dos tempos idle.
CPU	`esp_task_count`	Quantidade atual de tarefas FreeRTOS.
CPU adicional	`esp_cpu_temperature_celsius`	Temperatura interna do chip via `temperatureRead()`, somente se finita; nao e temperatura ambiente.
Clima adicional	`esp_weather_data_age_seconds`	Segundos desde a ultima atualizacao valida do clima atual Open-Meteo.
Memoria	`esp_heap_free_bytes`	Heap livre.
Memoria	`esp_heap_total_bytes`	Heap total.
Memoria	`esp_heap_minimum_free_bytes`	Menor heap livre desde boot.
Memoria	`esp_psram_free_bytes`	PSRAM livre; zero quando nao disponivel.
Memoria	`esp_psram_total_bytes`	PSRAM total; zero quando nao disponivel.
GPIO	`gpio_digital_input`	Botao BOOT GPIO0, label `pin`: 0 pressionado, 1 solto; configurado como entrada com pull-up.
GPIO	`gpio_digital_output`	Nivel logico do LED GPIO4, label `pin`: neste hardware o LED acende em LOW (0).
Sistema	`esp_reboot_count`	Inicializacoes acumuladas menos a primeira; inclui ciclos de energia.
Sistema	`esp_watchdog_resets`	Resets por watchdog geral, de interrupcao e de tarefa, acumulados na NVS.
Sistema	`esp_free_rtos_tasks`	Mesma quantidade de tarefas de `esp_task_count`.
Sistema	`esp_flash_usage_percent`	Tamanho do firmware / tamanho da particao em execucao, em porcentagem; nao e uso de toda a flash.

Grupo	Metrica	Semantica
Aplicacao	`application_requests_total`	Counter de requisicoes HTTP concluidas nos dois servidores.
Aplicacao	`application_errors_total`	Counter de respostas HTTP 4xx/5xx ou sem codigo de resposta.
Aplicacao	`application_response_time_ms`	Gauge do tempo de tratamento e envio da ultima requisicao concluida.

Limitacoes e disponibilidade

- Uso de CPU requer estatisticas FreeRTOS de runtime e duas amostras validas separadas por cinco segundos. A serie e omitida durante aquecimento, sem suporte do SDK, acima de 64 tarefas ou quando a amostra nao e valida. ESP-IDF usa microssegundos; a capacidade e normalizada por todos os nucleos.
- A idade do clima e omitida antes da primeira leitura atual valida. Usa `esp_timer_get_time()` de 64 bits, sem depender de NTP ou da virada de `millis()`. Uma falha de atualizacao preserva o cache e a idade continua crescendo; uma leitura atual valida reinicia a idade mesmo que a previsao diaria falhe. Mede tempo desde a recepcao, nao a idade da observacao meteorologica na origem.
- GPIO sao niveis instantaneos, nao contagem de pulsos. SD, TFT e touch nao sao reconfigurados para obter metricas GPIO.
- HTTP e desconexoes reiniciam no boot. O scrape atual so entra nos contadores apos terminar a resposta. Tempo HTTP nao inclui toda a latencia do cliente, nao e media ou percentil e nao contabiliza comandos UDP/serial.
- Dispositivo desligado nao publica zero: use `up` do Prometheus para detectar indisponibilidade. `esp_status_online` indica apenas a conexao STA.

Metricas do cartao SD

Pinagem TF conforme PINAGEM.md e SD_CARD.md: CS **5**, CLK **18**, MOSI **23**, MISO **19**. Sem alteracao da pinagem. A coleta copia somente campos numericos do cache sob o mutex SPI existente, sem montar, ler, gravar, trocar pinos ou aguardar uma operacao de arquivo. Contadores de logs sao atomicos para uso pelas tarefas de clima e aplicacao.

Metrica	Tipo	Semantica
`sdcard_metrics_available`	gauge	1 quando foi possivel copiar o cache; 0 se o mutex estiver ocupado ou nao inicializado.
`sdcard_mounted`	gauge	Estado de montagem/prontidao conhecido: 1 pronto, 0 ausente ou com falha. Nao e um teste fisico a cada scrape.
`sdcard_capacity_bytes`	gauge	Capacidade fisica informada por `cardSize()`.
`sdcard_total_bytes`	gauge	Capacidade do sistema de arquivos.
`sdcard_used_bytes`	gauge	Espaco ocupado no sistema de arquivos.
`sdcard_free_bytes`	gauge	Total menos usado, somente se usado <= total.

Metrica	Tipo	Semantica
`sdcard_usage_percent`	gauge	100 * usado / total, somente com total positivo e usado <= total.
`sdcard_spi_frequency_hz`	gauge	Frequencia de montagem selecionada: 4000000, 1000000 ou 400000 Hz.
`sdcard_log_size_bytes`	gauge	Tamanho conhecido de `/log.txt`.
`sdcard_log_records_total`	counter	Registros de log gravados com sucesso desde boot, nao o total de linhas historicas.
`sdcard_log_errors_total`	counter	Tentativas de registro perdidas/falhas desde boot, inclusive sem cartao; nao conta todas as operacoes SD.

Se nao for possivel copiar o cache, somente `sdcard_metrics_available=0` e publicada desse grupo; nao se inventa um estado de montagem. Se a copia for possivel mas o SD estiver indisponivel, publicam-se disponibilidade, montagem e contadores; capacidades, ocupacao, frequencia e tamanho de log sao omitidos. Espaco e atualizado pelo gerenciador a cada 30 segundos, na montagem ou em `sd_status`; tamanho do log tambem e atualizado nas gravacoes. A retirada do cartao pode ser percebida somente no proximo acesso/verificacao.

Metricas solicitadas ainda indisponiveis

Nao sao publicados zeros ficticios nem dados de API como se fossem sensores locais. Estas 17 series dependem de hardware/configuracao e instrumentacao:

Grupo	Metricas	Integracao necessaria
Energia	`esp_voltage_v`, `esp_current_ma`, `esp_power_w`	Medidor eletrico e calibracao.
Bateria	`esp_battery_percent`, `esp_battery_voltage_v`	Entrada de medicao, calibracao e curva da bateria.
Sensores locais	`sensor_temperature_celsius`, `sensor_humidity_percent`, `sensor_pressure_hpa`, `sensor_air_quality`	Sensores locais conectados. Temperatura/umidade Open-Meteo nao medem o ambiente da placa.
Reles	`relay_state`, `relay_activation_count`, `relay_runtime_seconds`	Mapeamento de reles e instrumentacao das transicoes.
ADC	`gpio_analog_value`	Pino e escala calibrada, sem conflito com os perifericos.
MQTT	`mqtt_connected`, `mqtt_messages_sent`, `mqtt_messages_received`, `mqtt_publish_errors`	Cliente MQTT e callbacks de conexao, publicacao e recepcao.

Monitoramento UDP

As metricas UDP usam o mesmo rotulo `device` das demais series:

Metrica	Tipo	Significado
`esp_udp_listener_up`	gauge	1 se a abertura da porta UDP funcionou e o Wi-Fi esta conectado; 0 caso contrario.
`esp_udp_packets_received_total`	counter	Datagramas identificados por `parsePacket`, incluindo comandos externos e respostas dos ESPs.
`esp_udp_replies_sent_total`	counter	Respostas a comandos externos cujo envio local terminou com sucesso.
`esp_udp_send_errors_total`	counter	Falhas ao enviar comandos ou respostas: IP invalido, inicio do pacote, escrita incompleta ou finalizacao.

Os contadores sao de 64 bits, protegidos entre nucleos, acumulam desde o boot e permanecem ao reabrir o socket/reconectar. Zere-os somente reiniciando a Central. Sucesso no envio UDP confirma a entrega ao socket local; nao e uma confirmacao do destinatario.

Compilacao e verificacao na placa

Mantenha `build_opt.h` junto ao sketch, ESP32 Dev Module, PSRAM desativada e **Minimal SPIFFS (1.9MB APP with OTA/128KB SPIFFS)**, conforme o projeto. FQBN: `esp32:esp32:esp32:PartitionScheme=min_spiffs`. SD nao utiliza o SPIFFS. Nao grave uma tabela de particoes diferente apenas por OTA do aplicativo.

O teste existente `node webpage.test.cjs` valida o painel e confere se o gzip corresponde ao HTML editavel; apos alterar WebPage.h, execute `node compress-web.cjs`. Para verificacao sintatica focalizada no ambiente configurado pelo projeto:

```
node check-firmware.cjs MetricsService.cpp WifiConfig.cpp SdManager.cpp
Central.ino CommandHandler.cpp
```

Isso nao substitui compilacao/link completos, limites de flash/RAM nem testes de rede e NVS na placa. Roteiro:

1. Em STA e AP, confira painel, log e formulario Prometheus na porta 80.
2. Salve host, porta 9121 e intervalo 20. Baixe o YAML e confira `:9121/20s`.
3. Apos um segundo, consulte `http://IP:9121/metrics` (200 e content-type Prometheus), `http://IP/metrics` (404) e painel na 80.
4. Reinicie e confira as preferencias restauradas, incremento de boots/reboots e watchdog sem incremento em reboot normal.
5. Troque para 80 e depois para 9121: apenas a porta ativa deve oferecer `/metrics`, mantendo painel/formulario na 80.
6. Tente host com protocolo, porta 0/65536 e intervalo 4/3601: espere 400 sem alterar a configuracao. Erros reais de NVS devem retornar 503.
7. Com SD, compare capacidades e log com `sd_status`/`sd_log`. Grave um comando e observe o contador. Sem SD pronto, espere `sdcard_mounted=0` e omissao de capacidades, sem prejudicar

touch/display/UDP. Mutex ocupado deve resultar em `sdcard_metrics_available=0`, nao falha de montagem simulada.

8. Confira GPIO0 ao pressionar BOOT durante funcionamento (nao durante reset), GPIO4 com `led_on`/`led_off`, temperatura com `temp` e CPU apos aquecimento.
9. Confira idade do clima ausente antes de sincronizar, crescente apos sucesso e preservada apos falha. Confira os contadores HTTP apos requisicao valida e rota inexistente.
10. Aplique o YAML no Prometheus e confira `up{job="esp32_central"} == 1`. Desligue a Central e confirme `up == 0`.